

and `aws_route53_record` to create `cname` records for the instances created. Every resource block takes up some arguments to customise the resource(s) it creates and exposes some attributes of the resource(s) created. Each resource block starts with `resource <resource type> <resource id>`, and the important thing is that the `<resource type> <resource id>` combination should be unique in the same Terraform configuration scope. The prefix of each resource is linked to its provider, e.g., all the AWS prefix resources require an AWS provider. The simple form of accessing the attribute of a resource is `<resource type>.<id>.<attribute>`. Our example shows that the `public_ip` and `public_dns` attributes of the created instances are being accessed in `route53` and `output` blocks.

Some of the resources require a few post-creation actions like connecting and running local and/or remote commands, scripts, etc, on AWS instance(s). The connection block is declared to connect to that resource, e.g., by creating a ssh connection to the created instances in our example. The provisioner blocks are the mechanisms to use the connection to upload file(s) and the directory to the created resource(s). The provisioners also run local or remote commands, while Chef runs concurrently. You could explore those aspects in detail on the official provisioners help page. Our example is uploading a provisioning script and kicking that off remotely over ssh to provision the created instances out-of-the-box. Terraform provides some meta-parameters available to all the resources, like the count argument in our example. The `count.index` keeps track of the current resource being created to reference that now or later, e.g., we are creating a unique name tag for each instance created, in our example. Terraform deducts the proper dependencies as we are referencing the attribute of `aws_instance` in `aws_route53_record`; so it creates the instances before creating their `cname` records. You could use `meta-variable depends_on` in cases where there is no implicit dependency between resources and you want to ensure that explicitly. The above-mentioned variables help the page provide detailed information about the meta-variables too.

The last block declared in our example configuration is the output block. As is evident by the name itself, the output could dump the raw or transformed attributes of the resources created, on demand, at any time. You can also see the usage of various functions like the `format` and the `element` in the example configuration. These functions transform the variables into other useful forms, e.g., the `element` function is retrieving the correct `public_ip` based upon the current index of the instances created. The official interpolation help page provides detailed information about the various functions provided by Terraform.

Now let's look at how to decipher the output being dumped when we invoke different phases of the Terraform workflow. We'll observe the following kind of output if

we execute the command `terraform plan -var 'num_instances="3"'` after exporting the `TF_VAR_aws_access_key` and `TF_VAR_aws_secret_key`, in the working directory where the first example config was created:

```
+ aws_instance.test.0
...
+ aws_instance.test.1
...
+ aws_instance.test.2
...
+ aws_route53_record.test.0
...
+ aws_route53_record.test.1
...
+ aws_route53_record.test.2
```

Plan: 6 to add, 0 to change, 0 to destroy.

If there is some error in the configuration, then that will come up in the plan phase only and Terraform dumps the parsing errors. You can explicitly verify the configuration for any issue using the `terraform validate` command. If all is good, then the plan phase dumps the resources it's going to create (indicated by the + sign before the resources' names, in green colour) to converge to the declared model of the infrastructure. Similarly, the Terraform plan output represents the resources it's going to delete in red (indicated by the - sign) and the resources it will update in yellow (indicated by the ~ sign). Once you are satisfied with the plan of resources creation, you can run `terraform apply` to apply the plan and actually start creating the infrastructure.

Our second example is to get you more comfortable with Terraform, and use its advanced features to create and orchestrate some non-trivial scenarios. The code `example2.tf` can be downloaded from http://opensourceforu.com/article_source_code/sept17/terraform.zip. It actually automates the task of bringing up a working cluster out-of-the-box. It brings up a configurable number of multi-disk instances from the cluster payload AMI, and then initiates a specific order of remote provisioners using `null_resource`, some provisioners on all the nodes and some only on a specific one, respectively.

In the `example2.tf` template, multiple `null_resource` are triggered in response to the various resources created, on which they depend. In this way, you can see how easily we can orchestrate some not-so-trivial scenarios. You can also see the usage of `depends_on` meta-variable to ensure a dependency sequence between various resources. Similarly, you can mark those resources created by Terraform that you want to destroy